Evaluation

The Virtual Cell Challenge evaluates submitted results from entrants' models against held-out measurements obtained from Arc Institute's laboratory testing, and generates a score for each entry that reflects the entry's performance. The task, the submission format and the scoring methodology are outlined below.

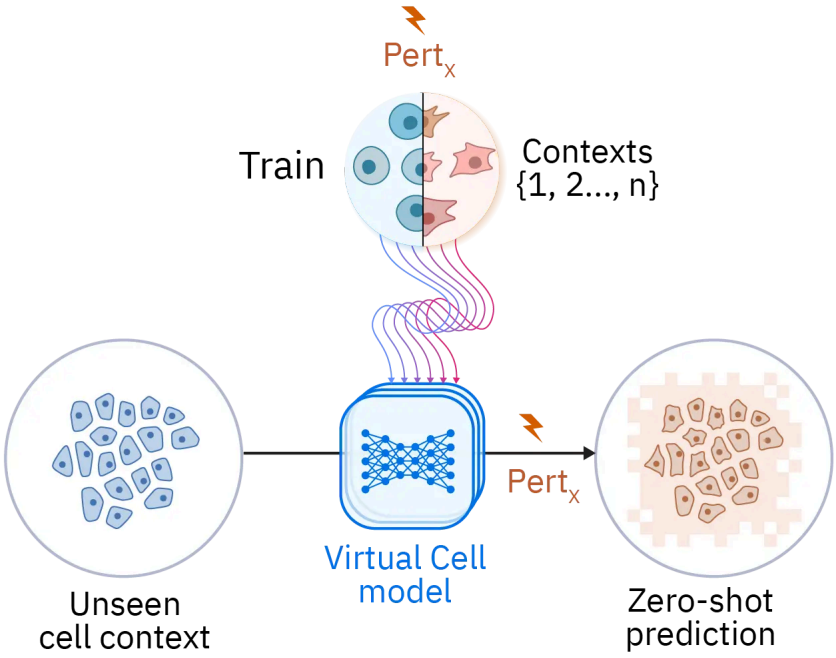
COMMENTARY IN CELL ↗

Task Submissions Scoring Resources

Task

Predictive models can be trained to generalize along several axes. Two core dimensions are (i) generalization across biological context (e.g. cell type, cell line, culture conditions, or even in vivo versus in vitro settings) and (ii) generalization to novel genetic and/or chemical perturbations, including their combinations.

The 2026 Challenge focuses on the first dimension, and raises it to multi-context generalization and zero-shot prediction. Arc performed Perturb-seq experiments on six cell lines originating from different tissues, using CRISPRi and 10x Flex scRNA-seq.



There is no Challenge training set this year. At the start of the Challenge you receive, for three of those cell lines, profiles from cells expressing non-targeting guides together with the gene identifiers for the 300 CRISPRi knockdowns to be predicted in each context. Your model must predict the post-perturbation expression profiles that would be measured after CRISPRi knockdown followed by 10x Flex single-cell profiling.

This design deliberately separates access to the target context from access to perturbation outcomes in that context. The non-targeting profiles define each line's basal state, the perturbation identities define the interventions to simulate, and the withheld expression profiles are the ground truth. A successful model has to infer how a perturbation response transfers across contexts without being given matched perturbation-response data from the Challenge itself.

The final test set follows the same structure in three different cell lines. Cell lines are never named — each is released only as an anonymized cell context, labelled A, B and C for validation and D, E and F for the final test.

Submissions

Predictions are submitted as `.vcc` files, produced by the vcc command-line tool. The `.vcc` format is a wrapper around an AnnData h5ad file that ensures the submission meets the requirements of the Challenge.

You build one h5ad covering all three cell contexts — not one file per context — and package it with vcc prep, which validates the format locally so problems surface in seconds rather than after a multi-gigabyte upload:

```
vcc prep prediction.h5ad -g gene_names.csv --perts pert_counts.csv -o prediction.vcc
vcc submit prediction.vcc -m "my model v3" --wait
```

If you use a coding agent (Claude Code, Codex, Gemini), vcc skill installs a skill that lets the agent drive this whole workflow for you. Install it, restart your agent session, then ask in plain language:

```
vcc skill install
> "download the VCC control data and make a submission"
```

Re-run `vcc skill install` after every CLI upgrade so the skill matches the installed version.

Each team may submit two results per day, renewing at midnight UTC. Only submissions that reach the scoring step count against that limit — usage errors and anything rejected before a score is saved are not counted. A team can have only one submission in flight at a time.

Your submission of an entry during the first phase of the Challenge constitutes consent for us to post your team name, the names of your team members, your organization and your model name, together with your scores, on the continuously updated public leaderboard. Your submission of an entry during the final phase constitutes the same consent for the final public leaderboard released when winners are announced.

During the Challenge, results appear on the live rankings leaderboard. In the final phase the final test set is released; final submissions are not shown on the leaderboard, and winners are announced after the Challenge closes.

Submission File Requirements

The CLI guide walks through generating a valid prediction file. `vcc prep` enforces each of the following:

- One file covering all three cell contexts. Every cell carries a `context` label in .obs, reused exactly as it appears in the control files you downloaded — `A`/`B`/`C` during the validation phase, `D`/`E`/`F` in the final phase.
- The perturbation column must be named `target_gene` in .obs, holding gene symbols (`ADNP`), not construct ids (`ADNP-1`).
- Exactly the perturbations listed in pert_counts.csv — no extras, none missing. An extra perturbation is rejected rather than ignored.
- Exactly 400 cells for every perturbation, in every context. The count is the same for all of them, and a different one is rejected.
- Exactly 18,533 genes in .var, in the order given by gene_names.csv.
- Raw counts in .X — non-negative, whole numbers, and finite. Fractional values are rejected. Scoring runs in counts space, so a log-normalized matrix cannot be scored. Do not apply `normalize_total` or `log1p`.
- No non-targeting rows. Every row must be one of the perturbations listed in pert_counts.csv; a submission containing rows labelled `non-targeting` is rejected. The control cells you downloaded are model inputs only — do not copy them into your prediction.
- At most 4,750,000,000 stored entries in .X — about 13,200 per cell at the standard 360,000-cell shape. This is a limit on density, not file size: predictions compress well, so a small file can still be far too dense. It counts the values your matrix stores, not the ones that happen to be non-zero, so explicitly-stored zeros count against it. A dense array stores all 18,533 genes for every cell — 6,671,880,000 entries, 1.40× the cap — and so is over the limit on its own whatever it contains. Store the prediction sparse, without explicit zeros.
- No single cell above 1,000,000 counts. A cell's counts summed across all genes must not exceed 1,000,000 (`max_counts_per_cell`). Nothing about such a matrix looks wrong — the values are still whole and finite — so a too-large library size is easy to emit by accident; normalize each cell to a library size at or below the cap.

Prediction: prediction.h5ad | 360,000 n_obs (cells) × 18,533 n_vars (genes)

300 perturbations × 400 cells × 3 cell contexts. The perturbations come from `pert_counts.csv`; every one of them gets the same 400 cells.

Obs

target_gene	context
ABCD1	A
ABCD1	B

ACLY	A
...	...

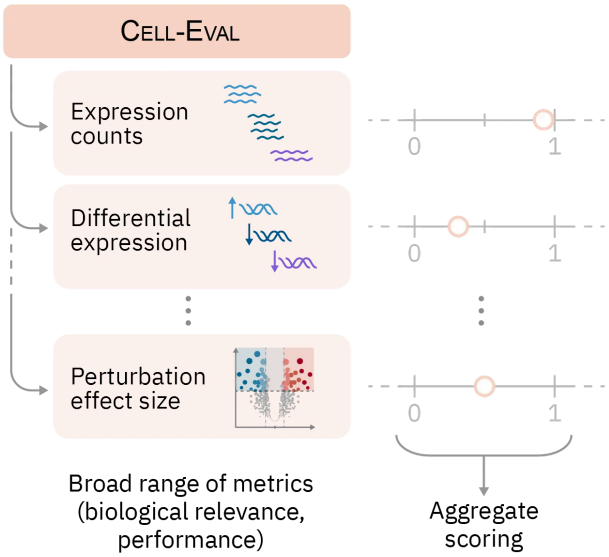
Var — index of gene names to predict, in the order given by gene_names.csv

```
Index: [SAMD11, NOC2L, KLHL17, PLEKHN1, PERM1, HES4, ISG15, AGRN, RNF223, C1orf159, TTLL10, TNFRSF18, TNFRSF4, SDF4, B3GALT6,
C1QTNF12, UBE2J2, SCNN1D, ACAP3, PUSL1, ...]
```

Scoring

Evaluation metrics should reflect the core purpose of a virtual cell: simulating cellular behavior in silico that is usually determined experimentally. This Challenge is focused on predicting gene expression responses to genetic perturbations, which reflects the most common readout from single cell functional genomics experiments: post-perturbation expression counts, and the differentially expressed genes derived from them. You submit predicted counts; we compute the differential expression.

Scoring is run by `cell-eval2` on its `vcc2026` metric profile. The Challenge scores six complementary metrics — one on expression, one on discrimination, and four on differential expression.



Based on our experience with the inaugural 2025 Challenge, the metrics for VCC 2026 have been thoroughly revised to be more robust, more comprehensive, and reference-scaled. Three ideas guided the revision: score the part of a measurement that replicates; scale each metric against the experiment itself rather than against perfection, with 0 at the cell context mean and 1 as good as data from a real replicate experiment; and measure the capability that transfers, zero-shot, into cell contexts the model has not seen. These metrics are new and we expect to keep refining them from year to year. More information on the math and motivation behind them will follow in a few weeks.

A major part of our motivation for the Virtual Cell Challenge is to help the scientific community develop high quality biologically-relevant evaluation metrics for emerging AI models. We hope this year's metrics are a step forward, but we expect to learn from the competition and the results and continue to improve on evaluation annually.

The six scored metrics

The perturbed gene's own row is excluded from all six: reporting that the knocked-down gene changed is the premise of the experiment rather than a prediction.

The short name beside each is the column header used on the leaderboard and in `vcc status`.

These descriptions are shorthand summaries; for the exact specifications on how we will score your entries, please see the vcc2026 metric specification.

Perturbation discrimination `pds`	Is the predicted effect closer to its own real effect than to any other perturbation's? Ranks every real effect by cosine distance from the prediction and reports where the true one lands. The rank is within the panel.
Expression accuracy `mse`	A squared distance between the predicted and real expression profiles, corrected for the sampling floor that comes purely from having a finite number of cells, then normalized by the size of the real effect. It is one ratio over the whole panel rather than a per-perturbation value.
DE log-FC accuracy `nmae`	Normalized mean absolute error of the log2 fold changes, over the genes the reference called significant — how close the predicted effect sizes are, not just their direction.
DE direction fidelity `fid`	Of the genes the prediction calls significant, the proportion moving the same direction as the real data — the real gene need not itself be significant. Scaled by yield, so calling few genes well is not enough.
DE direction reach `reach`	How deep the directions stay pure when the genes the reference called significant are ranked by the prediction's own confidence. Takes the deepest cut whose directional agreement holds above a threshold, as a fraction of the reference's own confident calls.
DE significance overlap `jac`	Of all the genes either side called differentially expressed, the fraction both called. Divides by the union, so over-calling is penalized.

Overall Score
Every measurement carries experimental noise, so the useful question is not how close a prediction comes to perfection but how it compares to the experiment itself. Each metric is therefore reference-scaled against two points measured in its own cell context — the cell context mean, which predicts the same average profile for every perturbation, and data from a real replicate experiment, taken as the mean of five disjoint half-splits of the real data. A metric is placed between them, so for every metric and every context:

- `0` — no better than the cell context mean
- `1` — as good as data from a real replicate experiment
- `above 1` — better than data from a real replicate experiment
- `below 0` — worse than the cell context mean; how far below depends on the metric

The overall score is a single unweighted average over the six metrics and the three cell contexts — nothing is weighted by perturbation count or cohort size. Scaling each context against its own two reference points is what makes that flat average meaningful: `0` and `1` mean the same thing in every context.

> 1.0 is not the maximum, and this is not a percentage. Only expression accuracy (`mse`) clamps at the top, and it is the one metric bounded at both ends, on 0 to 1. The other five are unbounded above, so a strong submission can score above 1. Four of the six are also unclamped below and reach different depths, so there is no single floor either. Negative scores are normal rather than an error — below 0 simply means worse than the cell context mean. Never read the score as "% correct" or as a fraction of an achievable range; there is no fixed range to take a fraction of.

Validation and the final test are scored independently and are not comparable — different cell lines, different perturbation panels, and perturbation discrimination is a rank *within* its own panel. A lower final phase score than your validation score does not mean your model got worse.

Resources

We have generated some resources that may assist participants in the Challenge.

vcc CLI guide ↗
The participant guide for the `vcc` command-line tool: installing it, authenticating, downloading the control data, packaging a prediction with `prep`, submitting, and checking your score — plus a full command reference.

cell-eval2 ↗
The scorer itself — the same code that scores your submissions, on its `vcc2026` metric profile.

The written specification for the six metrics lives in the same repository: the vcc2026 metric specification.

State: Arc's own Virtual Cell Model ↗

Source code for State, Arc's initial virtual cell model trained on datasets from Arc Virtual Cell Atlas and elsewhere (this model is intended as a starting benchmark and is not able to win the competition).

Please note that trained versions of State are subject to a noncommercial license, which you can find here.